

Hybrids:

the evolution of $\text{CON}\text{T}_{\text{E}}\text{X}_{\text{T}}$

- Coding in $\text{T}_{\text{E}}\text{X}$ is quite natural and given a proper macro set the overhead is not that large.
- Coding in xML makes sense when you have to manipulate or reuse your data and when $\text{T}_{\text{E}}\text{X}$ is just the renderer.
- For non-artistic graphics MetaPost provides a convenient input language. It also plays well with $\text{T}_{\text{E}}\text{X}$.
- Some problems can more conveniently be solved in a procedural programming language and LUA perfectly fits in there.

- Of course we started with only $\text{T}_{\text{E}}\text{X}$ code. Functionality has been nicely split in modules
- Front- and backend code has always been separated.
- The user interface is quite consistent which provides backward compatibility as well extensibility.
- For quite some time MetaPost support has been tightly integrated, including a two way communication between these subsystems.
- When we decided on LUA as language it didn't take long before large chunks of $\text{CON-T}_{\text{E}}\text{X}_{\text{r}}$ were rewritten using it.

- Most font handling takes place in `LUA` and as usual with `TEX` we can do more than fonts provide.
- Other subsystems, like languages, input encoding, file io and xml also were among the first to be supported by `LUA`.
- Lots of information is now carried around, especially related to structure. This will permit users more freedom.
- Notes, descriptions and enumerations also rely on `LUA`.
- Graphics (including `MetaPost`) is all dealt with in `LUA`. Float management is currently on the agenda.
- The backend code is completely rewritten in `LUA`. We've disabled the low level primitives so that third party modules can not spoil the game (this was already the case in `MkII`).
- Eventually most management tasks will move from `TEX` to `LUA`, but we keep in pace with `LUATEX` development and don't push things to the limit.

- Eventually we will have a more layered macro package so that one can make specialized versions.
- In addition to the regular $\text{\TeX}$ interface there will be a $\text{\Lua}$ interface. We already have one such interface but there will be more.
- It will be possible to avoid $\text{\TeX}$ code completely which makes sense in predictable workflows where no artistic intervening is needed.
- Core $\text{\TeX}$ functionality will also be available as (often less efficient) $\text{\Lua}$ variant so that we can extend it. We already provide hooks into the callback subsystem.
- We can already all of this intermixed so the user has complete freedom of choice.